Route Architecture

How Shipments Get Their Routes — Across All Patterns

Load = the operation (WHAT to move)
Shipment = the truck + its route (HOW to move it)

Routes belong to shipments. Smart defaults make standard loads feel like shared routes. Transfer stops signal cross-dock independence.

March 2026 | Architecture Decision

The Two Layers

LOAD = BUSINESS LAYER (WHAT)

→ Which PRs are we fulfilling?
→ What's the ultimate destination?
→ Documents, invoices, commercial value
→ Pattern label (auto-detected)
→ The "operation" or "job"

SHIPMENT = EXECUTION LAYER (HOW)

→ Which truck? (vehicle, driver)
→ Which route? (its OWN ordered stops)
→ Each stop: location, action, planned/actual qty
→ Status: draft → planned → in transit → completed
→ The "trip"

> **Key principle:** Routes belong to shipments, not loads. But for standard loads (90% of cases), the system auto-generates and syncs routes so it FEELS like a shared route. For cross-dock (10%), each shipment's route is independently managed.

4 Ways a Route Gets Created

1 Auto-Generated at Load Creation

User selects PRs → Create Load → System auto-creates shipment with route: PICKUP per PR + DELIVER at destination. Zero manual work.

```
User selects: PR-001 (Daikin, Ahmedabad), PR-002 (Deloitte, Hyderabad), PR-003
(Infosys, Pune)
Destination: Haridwar Plant

System creates:
  Shipment 1 (auto):
    Stop 1: PICKUP → Daikin, Ahmedabad       (from PR-001)
    Stop 2: PICKUP → Deloitte, Hyderabad     (from PR-002)
    Stop 3: PICKUP → Infosys, Pune           (from PR-003)
    Stop 4: DELIVER → Haridwar Plant         (from destination)
```

2 Cloned When Adding a Shipment (+ Add Truck)

User clicks "+ Add Shipment" → System clones the ENTIRE route from the first shipment. New truck, same route. User can edit after.

```
User clicks "+ Add Shipment"

System creates:
  Shipment 2 (cloned from Ship 1):
    Stop 1: PICKUP → Daikin, Ahmedabad       (same)
    Stop 2: PICKUP → Deloitte, Hyderabad     (same)
    Stop 3: PICKUP → Infosys, Pune           (same)
    Stop 4: DELIVER → Haridwar Plant         (same)

Both trucks now follow the same route.
Material split happens at pickup (reality, not planning).
```

3 Synced When Adding a PR (+ Add Source)

User adds PR-004 to the Load → System adds a PICKUP stop to ALL existing shipments. Routes stay in sync.

```
User adds PR-004 (Mitsubishi, Chennai) to the Load

System updates ALL shipments:
  Shipment 1: ... + Stop 5: PICKUP → Mitsubishi, Chennai
  Shipment 2: ... + Stop 5: PICKUP → Mitsubishi, Chennai

Both trucks now visit 4 clients instead of 3.
Sync = automatic for standard loads.
```

4 Manually Built by User (Cross-Dock)

User adds TRANSFER stops → that shipment becomes independently routed. No auto-sync. User builds the route from scratch.

```
User creates cross-dock manually:

  Shipment 1 (Feeder):
    Stop 1: PICKUP → Jaipur client
    Stop 2: TRANSFER_OUT → location TBD (will meet big truck)

  Shipment 2 (Feeder):
    Stop 1: PICKUP → Ajmer client
    Stop 2: TRANSFER_OUT → location TBD

  Shipment 3 (Line-Haul):
    Stop 1: TRANSFER_IN → from Ship 1 + Ship 2
    Stop 2: DELIVER → Haridwar Plant

Each route is INDEPENDENT. No sync between shipments.
```

The Sync Rule — When It Applies, When It Doesn't

AUTO-SYNC Mode (Default)

When a shipment has ONLY standard stops (PICKUP + DELIVER), its route is auto-synced with other shipments in the load. Add a PR → all trucks get it. Add a truck → clones the route.

INDEPENDENT Mode (Cross-Dock)

When a shipment has ANY transfer stop (TRANSFER_IN or TRANSFER_OUT), it exits auto-sync. Its route is independently managed. Adding a PR to the load does NOT auto-add a pickup to this shipment.

> **The trigger is implicit:** User doesn't flip a switch. They just add a TRANSFER stop, and the system knows this shipment needs independent routing. No "cross-dock mode" button.

How the System Decides

```
function isShipmentSynced(shipment, stops):
  shipmentStops = stops.filter(s => s.shipmentId === shipment.id)
  hasTransfer = shipmentStops.some(s =>
    s.type === 'TRANSFER_IN' || s.type === 'TRANSFER_OUT'
  )
  return !hasTransfer  // synced if NO transfer stops
```

SHIPMENT HAS	SYNC MODE	BEHAVIOR
PICKUP + DELIVER only	Auto-Sync ✅	Add PR → synced. Add Ship → cloned.
PICKUP + TRANSFER_OUT	Independent 🔒	Feeder — route managed separately.
TRANSFER_IN + DELIVER	Independent 🔒	Line-haul — route managed separately.
TRANSFER_IN + PICKUP + DELIVER	Independent 🔒	Cross-dock + milk run combo.

Every Action — Full Matrix

Standard Loads (Auto-Sync)

ACTION	WHAT HAPPENS	ROUTE IMPACT
+ Add PR	New PR added to Load	**PICKUP stop added to ALL synced shipments**
+ Add Shipment	New truck created	**Route CLONED from first synced shipment**
- Remove PR	PR removed from Load	**PICKUP stop removed from ALL synced shipments**
- Remove Shipment	Truck removed	Other routes unchanged
Reorder Stops	User reorders on one shipment	Per-shipment. Each truck can have different order.
Remove Stop from ONE ship	User removes a pickup from one truck	Only that shipment affected. Route diverges.
Change Destination	Load destination updated	**DELIVER stop updated on ALL shipments**

Cross-Dock (Independent)

ACTION	WHAT HAPPENS	ROUTE IMPACT
+ Add PR	PR added to Load	User CHOOSES which feeder gets the pickup
+ Add Feeder	New feeder shipment	Empty route — user builds it (PICKUP + TRANSFER_OUT)
+ Add Line-Haul	New line-haul shipment	Empty route — user builds it (TRANSFER_IN + DELIVER)
Link Feeder → Line-Haul	TRANSFER_OUT ↔ TRANSFER_IN paired	Connection established
Transfer Location	Driver fills at execution (call coordination)	Location TBD at planning. Filled at execution.

Natural Evolution — Milk Run to Cross-Dock

A user can organically evolve a load from simple to complex without switching modes:

1 **Start: Milk Run**

3 PRs, 1 shipment. Standard auto-sync.

```
Ship 1: PICKUP Jaipur → PICKUP Ajmer → PICKUP Jaisalmer → DELIVER Plant
Mode: Auto-Sync ✅
```

2 **Realize: Jaisalmer is too far for the same truck**

User decides to split: main truck does Jaipur + Ajmer, separate feeder picks Jaisalmer.

3 **User removes Jaisalmer from Ship 1**

Ship 1 route: PICKUP Jaipur → PICKUP Ajmer → DELIVER Plant

4 **User adds Ship 2 with PICKUP Jaisalmer + TRANSFER_OUT**

Ship 2 becomes independent (has TRANSFER stop). It's now a feeder.

5 **User adds TRANSFER_IN to Ship 1 (before DELIVER)**

Ship 1 also goes independent. It now receives from Ship 2 before delivering to plant.

```
Final state:

Ship 1: PICKUP Jaipur → PICKUP Ajmer → TRANSFER_IN (from Ship 2) → DELIVER Plant
  Mode: Independent 🔒 (has TRANSFER_IN)

Ship 2: PICKUP Jaisalmer → TRANSFER_OUT (to Ship 1, location TBD)
  Mode: Independent 🔒 (has TRANSFER_OUT)

Load evolved from Milk Run → Cross-Dock without any mode switch.
Pattern auto-detected: cross_dock
```

> **No modal, no wizard, no "create cross-dock" button.** The user just edits stops, and the system adapts. TRANSFER stops are the signal.

Transfer Stops — The Cross-Dock Primitives

TRANSFER_OUT (Feeder gives material)

FIELD	AT PLANNING	AT EXECUTION
Location	TBD (optional)	Filled by driver (phone coordination)
Linked to	**Target shipment ID (line-haul)**	Same
Actual Qty	—	Recorded at transfer

TRANSFER_IN (Line-haul receives material)

FIELD	AT PLANNING	AT EXECUTION
Location	TBD (auto-fills from linked TRANSFER_OUT)	Same location as TRANSFER_OUT
Linked to	**Source shipment ID(s) (feeders)**	Same
Actual Qty	—	Auto-populated from linked TRANSFER_OUTs
Blocked until	—	All linked TRANSFER_OUTs completed

> **Linking mechanism:** TRANSFER_OUT.linkedStopId → TRANSFER_IN.id (and vice versa). One TRANSFER_IN can receive from multiple TRANSFER_OUTs. Location on TRANSFER_IN auto-fills when the first TRANSFER_OUT records its location.

Readiness Check — Cross-Dock Shipments

SHIPMENT TYPE	CAN PLAN WITHOUT	REQUIRED FOR DISPATCH
Standard	—	Vehicle + Driver + All stop locations
Feeder	Transfer location (TBD OK)	Vehicle + Driver + Pickup location
Line-Haul	Transfer location (TBD OK)	Vehicle + Driver + Deliver location

> **Transfer location is NOT required for planning/dispatch.** It's filled at execution time when drivers coordinate. This matches reality — "I'll meet you somewhere on the highway, will confirm location."

Complete Pattern Coverage

PATTERN	SOURCES	SHIPS	ROUTE TYPE	HOW ROUTE CREATED
Direct	1 PR	1	Auto-Sync	Auto: 1 pickup + 1 deliver
Milk Run	2+ PRs	1	Auto-Sync	Auto: N pickups + 1 deliver
Multi-Vehicle	any PRs	2+	Auto-Sync	Cloned: same route, multiple trucks
Warehouse Collection	PRs	any	Auto-Sync	Auto: pickups + deliver to WH
Warehouse Dispatch	WH items	any	Auto-Sync	Auto: pickup from WH + deliver to plant
Cross-Dock	PRs	2+	Independent	Manual: user adds TRANSFER stops
Cross-Dock + Milk Run	PRs	2+	Independent	Manual: TRANSFER + PICKUP on line-haul

ONE data model. ONE architecture. TWO behaviors (auto-sync vs independent). The system adapts based on what the user does, not what mode they select.

Pattern Auto-Detection Logic

```
function detectPattern(load, shipments, stops):
  hasTransfer = stops.some(s =>
    s.type === 'TRANSFER_IN' || s.type === 'TRANSFER_OUT'
  )

  if (hasTransfer):
    return 'cross_dock'  // or cross_dock_milk_run if line-haul has PICKUPs

  hasWarehouseDest = load.destination.type === 'warehouse'
  hasWarehouseSource = load has warehouse source items

  if (hasWarehouseSource): return 'warehouse_consolidation'
  if (hasWarehouseDest): return 'warehouse_consolidation'

  prCount = load.prIds.length
  shipCount = shipments.length

  if (prCount === 1 && shipCount === 1): return 'direct'
  if (prCount >= 2 && shipCount === 1): return 'milk_run'
  if (shipCount >= 2): return 'multi_vehicle'  // covers milk_run_multi_vehicle too
```

Data Model — No Changes Needed

The current data model already supports this. Stops belong to Shipments. No structural changes required. Only behavior changes (sync logic + TRANSFER stop support).

What Gets Added (Code Only)

COMPONENT	CHANGE
`addShipmentToLoad()`	**Clone route from first synced shipment (instead of just first PR)**
`addPRsToLoad()`	**Add PICKUP to all SYNCED shipments (skip independent ones)**
`removePRFromLoad()`	**Remove from all SYNCED shipments (skip independent ones)**
`isShipmentSynced()`	NEW: check if shipment has transfer stops
Stop Type	TRANSFER_IN + TRANSFER_OUT already in types
FlowLayout	**Already handles 5-column cross-dock layout**
ShipmentExpandPanel	**Add "Feeds into" / "Receives from" display**

What Stays Exactly The Same

COMPONENT	STATUS
PR entity	No change
Load entity	No change
Shipment entity	No change
Stop entity	No change (TRANSFER types already exist)
Cascade engine (completeStop)	No change
PR List, Shipment List, Dashboard	No change
Warehouse Dashboard	No change

Summary — One Architecture, All Patterns

Data Model

PR → Load → Shipment → Stop. Unchanged. Already correct.

Route Ownership

Routes belong to Shipments. Each shipment has its own ordered stops.

Smart Defaults (90% of loads)

Auto-generate routes. Sync across trucks. Clone on add. Feels like shared route.

Cross-Dock (10% of loads)

TRANSFER stops = independent routing. No sync. User builds each route. Location TBD at planning, filled at execution.

Organic Evolution

Loads can evolve from simple → complex without mode switches. Add TRANSFER stop = becomes cross-dock. Remove it = back to standard. System adapts.

Zero data model changes. Zero new entities. Just smarter behavior on the SAME architecture.

Attero TMS — Route Architecture | March 2026